

Tema

Aplicação Node.js com Express para consulta de concursos da Mega-Sena.

O projeto utiliza uma base local de resultados da Mega-Sena, carregada em um banco PostgreSQL a partir de um arquivo CSV. Os dados foram obtidos por download na página oficial da CAIXA:

<https://loterias.caixa.gov.br/Paginas/Mega-Sena.aspx>

A aplicação disponibiliza uma página web simples para consultar o último concurso ou buscar um concurso específico pelo número. O frontend consome as rotas HTTP do servidor e apresenta as dezenas sorteadas, data do sorteio, premiações e estimativa do próximo prêmio.



Mega-Sena
Consulte o último resultado ou busque por concurso.

Concurso

3001 **Buscar**

Concurso 3001
28/04/2026 **Acumulou**

01 13 32 36 43 60

6 acertos
0 ganhador(es), R\$ 0,00

5 acertos
92 ganhador(es), R\$ 41.209,18

4 acertos
5877 ganhador(es), R\$ 1.063,34

Estimativa
R\$ 130.000.000,00

Requisitos de Avaliação

- Valor: 0,25 pontos na média final da disciplina.
- Forma de entrega: atividade **individual**. O estudante deverá **apresentar a aplicação em funcionamento**, publicada em um serviço de hospedagem em nuvem (por exemplo, Render ou outro serviço equivalente).
- Data de entrega: 15/05/2026.

Material de Apoio

A explicação sobre **a construção da aplicação** pode ser encontrada nos seguintes vídeos:

Configuração do servidor: https://youtu.be/ZuSc_GGEeE

Consulta no BD: <https://youtu.be/WRw-736gyUg>

Codificação da parte estática da página: https://youtu.be/_Gg01m7SRqw

Código do arquivo `.env`:

```
PORT=3005

POSTGRES_HOST=localhost
POSTGRES_USER=postgres
POSTGRES_PASSWORD=123
POSTGRES_DB=bdaula
POSTGRES_PORT=5432
```

Código do arquivo `src/infra/init/schema-sql.sql`:

```
CREATE TABLE IF NOT EXISTS public.megasena (
    concurso INTEGER NOT NULL,
    data_do Sorteio DATE NOT NULL,
    bola1 INTEGER NOT NULL,
    bola2 INTEGER NOT NULL,
    bola3 INTEGER NOT NULL,
    bola4 INTEGER NOT NULL,
    bola5 INTEGER NOT NULL,
    bola6 INTEGER NOT NULL,
    ganhadores_6_acertos INTEGER NOT NULL,
    cidade_uf VARCHAR(510) NULL,
    rateio_6_acertos DECIMAL NOT NULL,
    ganhadores_5_acertos INTEGER NOT NULL,
    rateio_5_acertos DECIMAL NOT NULL,
    ganhadores_4_acertos INTEGER NOT NULL,
    rateio_4_acertos DECIMAL NOT NULL,
    acumulado_6_acertos DECIMAL NOT NULL,
    arrecadacao_total DECIMAL NOT NULL,
    estimativa_premio DECIMAL NOT NULL,
    acumulado Sorteio Especial mega da virada DECIMAL NOT NULL,
    observacao VARCHAR(255) NULL,
    PRIMARY KEY(concurso)
);
```

Código do arquivo `src/infra/init/seed-sql.sql`:

```
TRUNCATE TABLE megasena;

COPY megasena
FROM '___SEED_DATA_DIR___/megasena.csv'
WITH (
    FORMAT csv,
    HEADER true,
    DELIMITER ';',
    NULL 'NULL',
```

```
    ENCODING 'UTF8'
  );
```

Código do arquivo `src/infra/run-sql.js`:

```
const fs = require("fs/promises");
const path = require("path");
const pool = require("../database/db");

const projectRoot = path.resolve(__dirname, "..", "..");
const sqlDir = path.resolve(__dirname, "init");
const seedDataDir = path.resolve(sqlDir, "seed-data");

// A ordem importa: esse array define a sequência exata de execução dos scripts.
const sqlFiles = [
  "schema-sql.sql",
  "seed-sql.sql"
];

async function runSqlFiles() {
  const files = sqlFiles.map((fileName) => path.join(sqlDir, fileName));

  if (files.length === 0) {
    throw new Error(`Nenhum arquivo .sql encontrado em ${sqlDir}`);
  }

  try {
    // Lê cada arquivo, ajusta caminhos locais e executa o SQL no banco configurado.
    for (const file of files) {
      let sql = await fs.readFile(file, "utf8");
      const relativeFile = path.relative(projectRoot, file).replaceAll(path.sep, "/");

      sql = prepareSql(sql);

      process.stdout.write(`Executando ${relativeFile}... `);
      await pool.query(sql);
      console.log("ok");
    }
  } finally {
    await pool.end();
  }

  console.log(`${files.length} arquivo(s) SQL executado(s).`);
}

function prepareSql(sql) {
  // O PostgreSQL espera caminhos com "/" no COPY, inclusive no Windows.
  const postgresPath = seedDataDir.replaceAll("\\", "/");

  // Substitui o placeholder usado nos .sql pelo caminho real dos arquivos CSV.
```

```
    return sql.replace(/__SEED_DATA_DIR__/g, postgresPath);  
  }
```

```
runSqlFiles().catch((error) => {  
  console.error("Erro ao executar arquivos SQL:");  
  console.error(error.message);  
  process.exitCode = 1;  
});
```

Código do arquivo `src/database/db.js`:

```
const dotenv = require("dotenv");  
const path = require("path");  
  
dotenv.config({  
  quiet: true,  
  path: path.resolve(__dirname, "..", "..", ".env"),  
});  
  
const { Pool } = require("pg");  
  
const config = {  
  host: process.env.POSTGRES_HOST,  
  user: process.env.POSTGRES_USER,  
  password: process.env.POSTGRES_PASSWORD,  
  database: process.env.POSTGRES_DB,  
  port: Number(process.env.POSTGRES_PORT || 5432),  
};  
  
const pool = new Pool(config);  
  
module.exports = pool;
```

Código do arquivo `src/routes/senas.routes.js`:

```
const { Router } = require("express");  
const { getConcurso, last } = require("../repositories/senas.repository");  
  
const routes = Router();  
  
routes.get("/", last);  
routes.get("/:concurso", getConcurso);  
  
module.exports = routes;
```

Código do arquivo `src/server.js`:

```
const dotenv = require("dotenv");  
const express = require("express");  
const path = require("path");
```

```
const router = require("./routes/senas.routes");

dotenv.config({
  quiet: true,
  path: path.resolve(__dirname, "..", ".env"),
});

const PORT = process.env.PORT || 3000;

const app = express();
app.use(express.json());

const publicPath = path.join(__dirname, "..", "public");
const pagesPath = path.join(publicPath, "pages");
const assetsPath = path.join(publicPath, "assets");

app.use("/", express.static(pagesPath));
app.use("/assets", express.static(assetsPath));
app.use("/api", router);

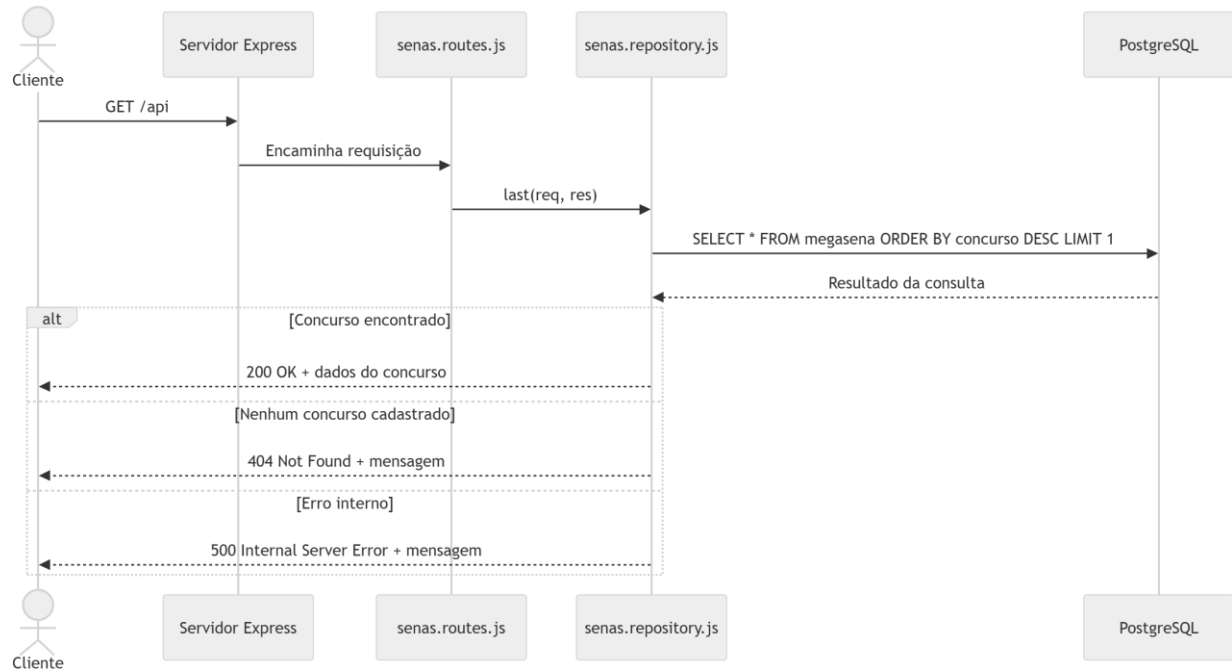
app.listen(PORT, function () {
  console.log(`Rodando em http://localhost:${PORT}`);
});
```

Código da propriedade scripts do arquivo `package.json`:

```
"scripts": {
  "start": "node ./src/server",
  "dev": "node --watch ./src/server",
  "db:init": "node ./src/infra/run-sql"
},
```

Diagramas de Sequência

Consulta do último concurso: GET /api



Consulta por número do concurso: GET /api/:concurso

